MetaMatch: A Topology-Aware Meta-Space for Schema Matching

Nour Elhouda KIRED*
nour-elhouda.kired@irit.fr
Université Toulouse II Jean Jaurès, IRIT
Toulouse, France

Olivier TESTE
olivier.teste@irit.fr
Université Toulouse II Jean Jaurès, IRIT
Toulouse, France

Abstract

Schema matching remains a central task in data integration. Recent embedding-based methods improve semantic comparison between columns, but they often rely on local similarities or distances between pooled column embeddings. In this paper, we propose MetaMatch, a supervised framework for complete tabular data based on a topology-aware meta-space. Each candidate column pair is represented by a meta-feature vector that combines syntactic, distance-based, and topological evidence extracted from transformer-based column representations. To our knowledge, such topological meta-features have not been used before for tabular schema matching. The topological component describes not only the source and target token clouds separately, but also their joint source–target representation in a shared embedding space. This joint view captures how the two columns merge geometrically, providing information that is not reduced to a single cosine distance or to two separate persistence diagrams. A supervised meta-learner then uses this meta-space to predict column correspondences. Experiments on the Valentine benchmark show that MetaMatch is competitive with established baselines, and that a reduced set of 10 meta-features preserves most of the effectiveness while substantially lowering runtime. The retained topological descriptor comes from the combined source–target representation, supporting the usefulness of joint topological meta-features for robust tabular schema matching.

Keywords

Schema Matching, Data Integration, Tabular Data, Meta-Space, Topological Data Analysis, Persistent Homology

PVLDB Reference Format:
Nour Elhouda KIRED and Olivier TESTE. MetaMatch: A Topology-Aware Meta-Space for Schema Matching. PVLDB, 19(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/NourKired/metamatch.

*Corresponding author.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License.
Visit https://creativecommons.org/licenses/by-nc-nd/4.0/.
Copyright is held by the author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 19, No. 1 (ISSN 2150-8097).
doi:XX.XX/XXX.XX

1 Introduction

Many real-world data integration tasks involve aligning different tabular sources. Aligning tabular data remains a complex and error-prone process in data integration, since different tables may describe the same concepts with different names, formats, or value distributions.

In practice, schema matching aims to identify correspondences between attributes that refer to the same concept in two tabular sources [9, 24]. This task remains challenging because tabular data are different at several levels. Column names may be short, ambiguous, or inconsistent across sources. Column values may be noisy, sparse, or strongly domain-specific. As a result, using similarity measures based solely on simple syntactic comparisons or shared-token signals is often insufficient. [16, 20, 25]

Traditional schema-based, instance-based, and hybrid methods compare labels, structural hints, data types, and value distributions [1, 7]. Supervised approaches combine several signals and learn how to separate matches from non-matches [2, 8, 22]. More recent methods rely on transformer-based language models to obtain vector representations of columns from embedding spaces [6, 13]. These methods are useful because they capture richer semantic relations. However, in many cases, the final comparison still relies on local measures computed between pooled vectors, such as cosine or Euclidean distance. [12, 29]

This local view does not fully exploit the internal structure of embedding representations. A column is not only represented by a single pooled vector. It is also represented by a set of contextual token embeddings produced by the encoder. These token-level representations contain geometric information that is largely lost after pooling. We believe that this internal structure can provide useful information for schema matching.

In this paper, we propose MetaMatch, a supervised framework for aligning complete tabular data. Our approach encodes columns into dense vector representations using a transformer-based sentence encoder enabling comparison within a shared representation space. From these embeddings, we construct a *meta-space* that captures hidden geometric relationships between candidate column pairs.

This meta-space is defined through a large set of meta-features. It includes syntactic meta-features computed from textual representations, distance-based meta-features computed from pooled embeddings, and topological meta-features computed from token-level embeddings. The syntactic and distance-based meta-features correspond to signals already used in the literature. The topological meta-features rely on an original geometric extraction based on persistent homology up to dimensions H_0, H_1, and H_2. An important part of this extraction is the construction of a joint source–target topological representation: instead of only computing one persistence diagram for each column and then comparing the two diagrams with a distance, we also stack the two token clouds in the

same embedding space and compute a single combined diagram. This combined diagram describes how both columns are organized when they are seen together in the same space.

To the best of our knowledge, this joint topological construction has not been explicitly used for schema matching on complete tabular data. Our objective is not to replace standard meta-features, but to enrich them with extra geometric information that cannot be reduced to a single distance between pooled embeddings. The feature-selection results later confirm the usefulness of this design, since the only topological descriptor retained in the reduced MetaMatch is a combined-cloud descriptor.

The main contributions of this work are as follows.

- We define a supervised schema matching framework dedicated to complete tabular data.
- We define a meta-space representation that combines 22 syntactic meta-features, 7 distance-based meta-features, and 31 topological meta-features computed from transformer-based embeddings.
- We introduce an original joint topological construction: source and target token embeddings are projected into the same representation space and summarized both through separate persistence diagrams and through a single combined source–target diagram.
- We provide a unified representation in which standard and topological signals are jointly exploited by a supervised classifier, and we identify a reduced 10-meta-feature core where the retained topological descriptor comes from the combined source–target construction.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 presents the proposed framework and details the meta-space generation. Section 4 reports the experimental evaluation. Finally, Section 5 concludes the paper and outlines future work.

2 Related Work

Schema matching is a long-standing problem in data integration [9, 24]. Classical approaches rely on lexical, structural, and instance-based evidence to identify correspondences between attributes. Representative systems such as COMA and COMA++ combine multiple matching signals in a flexible way [1, 7]. These methods remain important because they provide interpretable evidence and strong practical baselines. However, they are often sensitive to short labels, inconsistent naming conventions, noisy values, and weak metadata.

To address these limitations, supervised methods learn how to combine several signals from labeled examples. Early systems such as LSD and MARLIN showed that feature-based learning can improve matching decisions beyond hand-crafted rules [2, 8]. More recent approaches rely on neural representations and end-to-end learning [22]. In all these cases, however, the quality of the learned decision strongly depends on the quality of the input representation. If the feature space captures only a limited part of the relation between two columns, the learning stage can only exploit that limited information.

Recent advances in representation learning have further changed the landscape. Transformer models provide contextual embeddings that improve semantic comparison beyond exact lexical matching [6]. In matching-related tasks, such embeddings have been used to compare different labels and textual descriptions more effectively [13]. Nevertheless, most embedding-based approaches still describe a candidate pair through local pairwise measures computed on pooled vectors. Such measures are useful, but they do not describe the internal organization of token-level embeddings, and therefore ignore useful geometric structure.

Recent work has also explored schema matching with neural and large-language-model (LLM) pipelines. Embedding-based systems such as ISResMat show the benefit of fine-tuning pretrained encoders directly for relational schema matching [10]. Experimental evidence further shows that off-the-shelf large language models (LLMs) can assist schema matching, but that their effectiveness remains strongly dependent on the amount and quality of the provided context [23]. Other recent approaches improve matching through retrieval-enhanced prompting, generated semantic tags, unified LLM workflows, or combinations of small and large language models [19, 26, 27, 30]. These methods confirm the value of modern language models for schema matching, but they still mainly rely on semantic comparison, prompt design, or local similarity evidence rather than on an explicit description of the internal shape of embeddings.

This observation motivates the use of Topological Data Analysis (TDA). TDA studies the shape of data through tools such as persistent homology [3, 11, 31]. Instead of reducing a representation to a single point or a single distance, persistent homology captures how connected components, cycles, and higher-order structures appear and disappear across scales. Its outputs, namely persistence diagrams, can then be summarized through descriptors such as counts, maximum lifetimes, and persistence entropy, or compared through bottleneck and Wasserstein distances [4, 5, 21]. Although these tools have proved useful in other learning settings, we did not find prior work that explicitly uses topological extraction from token-level transformer embeddings to build a meta-space for schema matching on complete tabular data.

Our work is positioned at the intersection of these directions. We retain the standard syntactic and embedding-based signals used in schema matching, and we enrich them with topological descriptors derived from token-level embeddings in order to capture multi-scale structure that are lost after pooling.

3 Proposed Framework

3.1 Overview of the Framework

We consider two complete tabular sources

$$\mathcal{DS}_1 = (\mathcal{S}_1, \mathcal{I}_1), \qquad \mathcal{DS}_2 = (\mathcal{S}_2, \mathcal{I}_2),$$

where $\mathcal{S}_1 = [a_1, \ldots, a_n]$ and $\mathcal{S}_2 = [b_1, \ldots, b_m]$ denote the attribute lists, and $\mathcal{I}_1$ and $\mathcal{I}_2$ denote the instances. The task is to predict the alignment

$$\mathcal{A} = \{(u, v) \mid a_u \in \mathcal{S}_1 \text{ and } b_v \in \mathcal{S}_2 \text{ match}\}.$$

Here, $u \in \{1, \ldots, n\}$ indexes a source column in $\mathcal{DS}_1$, and $v \in \{1, \ldots, m\}$ indexes a target column in $\mathcal{DS}_2$. Thus, (u, v) denotes a candidate correspondence between source attribute a_u and target attribute b_v.

Figure 1 summarizes the framework. It contains four main stages: embedding generation, meta-space generation, meta-learner training, and alignment inference.

3.2 Detailed Framework Components

3.2.1 *Embedding Generation.*

Column serialization. For each column, we build a textual representation from the complete tabular information available in the source. More precisely, our implementation serializes each column as

$$x = \texttt{column\_name} \,\|\, \texttt{" || "} \,\|\, \text{Aggregate}(\texttt{column\_values})$$

where `column_name` is the attribute name in the table, $\|$ denotes ordered string concatenation with a fixed separator, and $\text{Aggregate}(\cdot)$ denotes a deterministic aggregation of representative values. In the current implementation, this aggregation keeps the most frequent non-empty values, with a fixed truncation budget. This gives a concise textual view of the column that contains both schema information and instance information. For the u-th source column and the v-th target column, we denote these inputs by x_1^u and x_2^v.

Transformer encoding. Each serialized column x is encoded by a transformer model. Let

$$Z(x) = \left[\mathbf{z}_1, \ldots, \mathbf{z}_{L_x}\right]^\top \in \mathbb{R}^{L_x \times d}$$

be the token-level embedding matrix returned by the encoder, where L_x is the number of tokens and d is the embedding dimension.

We use two representations.

- A pooled embedding vector

$$\mathbf{e}(x) = \frac{1}{L_x} \sum_{r=1}^{L_x} \mathbf{z}_r$$

- A token-level point cloud

$$X(x) = \{\mathbf{z}_r \mid 1 \le r \le L_x\} \subset \mathbb{R}^d$$

For a candidate pair (u, v), we write

$$\mathbf{e}_1^u = \mathbf{e}(x_1^u) \qquad \mathbf{e}_2^v = \mathbf{e}(x_2^v)$$

$$X_1^u = X(x_1^u) \qquad X_2^v = X(x_2^v)$$

The pooled vectors are used by the distance-based meta-features. The token clouds are used by the topological meta-features.

3.2.2 *Meta-Space Generation.*

The meta-space generation stage is the central block of the framework. It receives the serialized texts, pooled embeddings, and token-level embeddings associated with a candidate source-target column pair, and converts them into a single numerical vector. For each candidate pair (u, v), we construct a meta-feature vector

$$\mathbf{m}_{uv} = \phi_{\text{syn}}(u, v) \,\|\, \phi_{\text{dis}}(u, v) \,\|\, \phi_{\text{tda}}(u, v)$$

In the current version of the framework, the meta-space contains 60 meta-features: 22 syntactic meta-features, 7 distance-based meta-features, and 31 topological meta-features derived from persistent homology.

Classical Meta-Features. The classical part of the meta-space contains two families. These meta-features are already used in schema matching or in related text-similarity settings. In our framework, they are kept as strong evidence and as an essential component of the final representation.

(1) *Syntactic meta-features.* The syntactic meta-features correspond to the variables with prefix `syn_` in the implementation. They are computed from the serialized column texts after normalization by lowercasing, accent removal, separator normalization, and whitespace compression. The 22 meta-features cover string lengths, equality and containment indicators, edit distances and their normalized variants, longest-common-subsequence measures, common-prefix and common-suffix ratios, token-set similarities, character n-gram similarities, and Jaro/Jaro–Winkler similarities.
These meta-features provide explicit lexical evidence. They are especially useful when the two columns share a close naming convention or when representative values reuse the same terminology.

(2) *Distance-based embedding meta-features.* The distance-based meta-features are denoted with the prefix `dis_` in the paper. They are computed from the pooled embeddings $\mathbf{e}_1^u$ and $\mathbf{e}_2^v$. In this paper, we keep 7 geometric meta-features:
- Euclidean distance: `dis_euclidean`,
- cosine similarity: `dis_cosine_sim`,
- Pearson correlation: `dis_pearson`,
- Spearman correlation: `dis_spearman`,
- Minkowski distance with $p = 3$: `dis_minkowski`,
- Canberra distance: `dis_canberra`,
- Chebyshev distance: `dis_chebyshev`.

These meta-features capture local geometric relations in the embedding space. They remain simple, standard, and effective. For this reason, they form the distance-based part of the meta-space.

Topological Meta-Features. The topological part of the meta-space corresponds to the variables with prefix `tda_`. This is the original component of MetaMatch. While syntactic and distance-based meta-features compare columns through names, values, or pooled embeddings, the topological meta-features describe the geometry of the token-level embedding clouds.

From token embeddings to point clouds. For each serialized column text x, the transformer returns contextualized token embeddings. We interpret them as a point cloud

$$X(x) = \{\mathbf{z}_1, \ldots, \mathbf{z}_L\} \subset \mathbb{R}^d,$$

where each point corresponds to one token representation. A pooled embedding summarizes this cloud by a single vector. In contrast, persistent homology keeps information about how the token representations are organized across scales. This matters because two columns can have similar pooled embeddings while having different internal token structure.

For a point cloud X, we build a Vietoris–Rips filtration under the Euclidean metric. At scale $\varepsilon \ge 0$, the complex $\text{VR}_\varepsilon(X)$ connects points whose pairwise distances are small enough, and then fills higher-order simplices when groups of points are mutually connected. As ε increases, connected components merge, cycles may

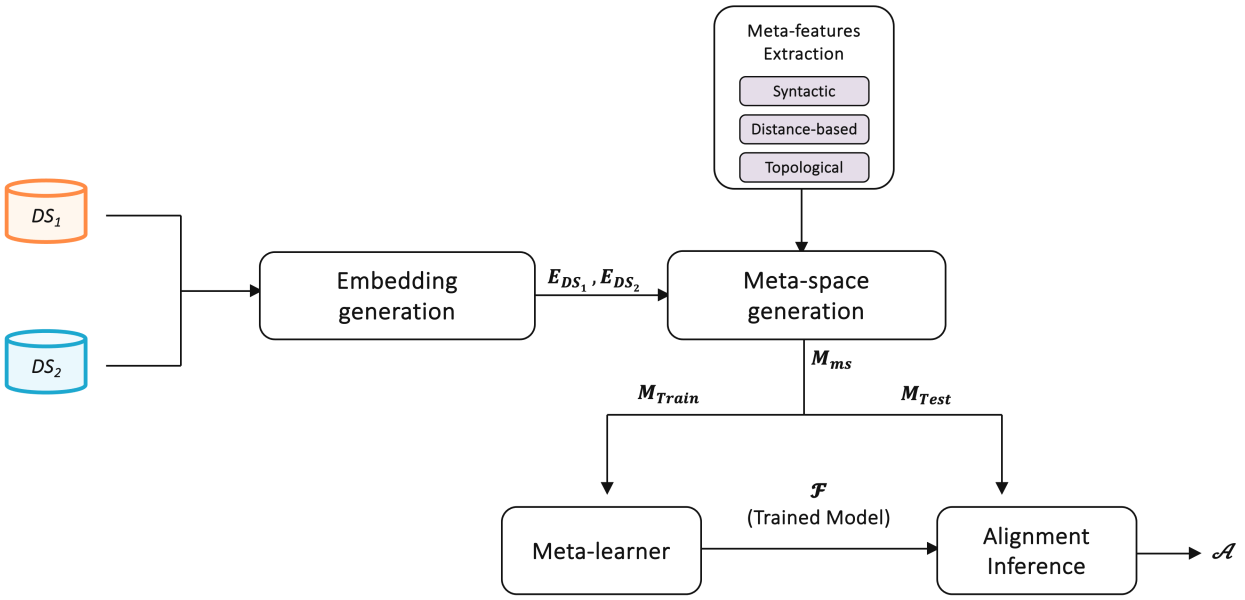

Figure 1: Overview of the proposed MetaMatch framework. Complete tabular sources are transformed into column embeddings. Candidate column pairs are represented in a meta-space built from syntactic, distance-based, and topological meta-features. A supervised meta-learner is trained on M_{Train} and applied to M_{Test} to infer the final alignment $\mathcal{A}$.

appear and disappear, and higher-dimensional voids may be formed. Persistent homology records these events through persistence diagrams.

We compute persistent homology up to dimension 2:

- H_0: connected components;
- H_1: one-dimensional cycles;
- H_2: two-dimensional voids.

Figure 2 illustrates the intuition on real candidate pairs.

For a homology dimension $\kappa \in \{0, 1, 2\}$, the persistence diagram is

$$\mathrm{Dgm}_\kappa(X) = \{(b_\tau^{(\kappa)}, d_\tau^{(\kappa)})\}_{\tau=1}^{N_\kappa},$$

where $b_\tau^{(\kappa)}$ and $d_\tau^{(\kappa)}$ are the birth and death scales of the τ-th structure. Its lifetime is

$$\pi_\tau^{(\kappa)} = d_\tau^{(\kappa)} - b_\tau^{(\kappa)}.$$

From each diagram, we extract three summaries:

$$\mathrm{count}_\kappa(X) = N_\kappa, \qquad \mathrm{maxlife}_\kappa(X) = \max_\tau \pi_\tau^{(\kappa)},$$

$$\mathrm{PE}_\kappa(X) = -\sum_{\tau=1}^{N_\kappa} p_\tau^{(\kappa)} \log p_\tau^{(\kappa)}, \qquad p_\tau^{(\kappa)} = \frac{\pi_\tau^{(\kappa)}}{\sum_{\mu=1}^{N_\kappa} \pi_\mu^{(\kappa)}}.$$

The count measures how many structures are detected, the maximum lifetime measures the most stable structure, and persistence entropy measures how persistence is distributed across structures.

Source, target, and joint topology. For a candidate source column u and target column v, let X_1^u and X_2^v be their token clouds. MetaMatch extracts topology at three levels. First, it describes the source cloud alone. Second, it describes the target cloud alone. Third, it constructs a joint source–target cloud in the same embedding space:

$$X_{uv}^{\mathrm{comb}} = \begin{bmatrix} X_1^u \\ X_2^v \end{bmatrix}.$$

Computing persistent homology on X_{uv}^{comb} gives one diagram for the candidate pair itself. This is different from only computing two diagrams separately and comparing them afterward. The joint diagram describes how the two token clouds behave when they are observed together. If the columns express the same concept, their clouds are expected to merge more coherently; if they express different concepts, the combined cloud can remain more separated or produce a different persistence profile.

Diagram distances. In addition to diagram summaries, we compare the source and target diagrams with bottleneck and Wasserstein distances. For two diagrams D_1 and D_2, the bottleneck distance is

$$d_B(D_1, D_2) = \inf_{\eta: D_1 \to D_2} \sup_{p \in D_1} \|p - \eta(p)\|_\infty,$$

and the Wasserstein distance is

$$W_2(D_1, D_2) = \left(\inf_{\eta: D_1 \to D_2} \sum_{p \in D_1} \|p - \eta(p)\|_\infty^2 \right)^{1/2},$$

where η ranges over matchings between diagram points, including matches to the diagonal. In MetaMatch, these distances are computed as

$$d_B\big(\mathrm{Dgm}_\kappa(X_1^u), \mathrm{Dgm}_\kappa(X_2^v)\big), \qquad W_2\big(\mathrm{Dgm}_\kappa(X_1^u), \mathrm{Dgm}_\kappa(X_2^v)\big),$$

for $\kappa \in \{0, 1\}$. We do not use H_2 diagram distances because finite H_2 structures, which correspond to voids, are often absent in short column token clouds. In that case, distances between H_2 diagrams are frequently undefined or uninformative. We therefore keep H_2 through stable per-cloud summaries, but restrict inter-diagram distances to H_0 and H_1.

Implemented topological meta-features. The final topological vector $\phi_{\mathrm{tda}}(u, v)$ contains 31 meta-features:

- 9 source-column meta-features: count, maximum lifetime, and entropy for H_0, H_1, and H_2;

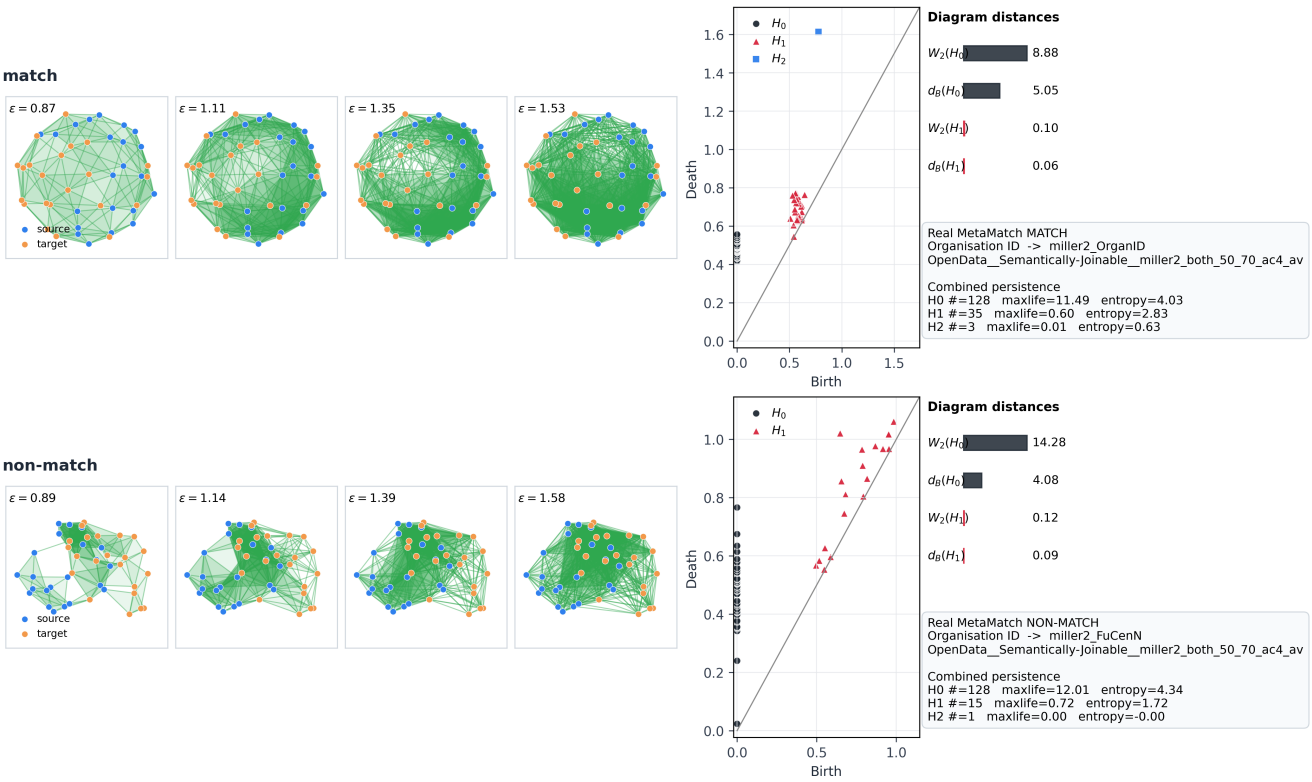

Figure 2: Vietoris–Rips filtration and persistence summary for a real matching and non-matching MetaMatch candidate pair from the same table pair. Each row shows the filtration, the persistence diagram up to H_2, the H_0/H_1 diagram distances, and the combined-cloud count, maximum lifetime, and entropy. The non-matching pair exhibits larger source–target diagram distances and a different persistence profile, illustrating why topological meta-features provide information beyond pooled embedding distance.

- 9 target-column meta-features: the same summaries for the target cloud;
- 9 combined-cloud meta-features: the same summaries for X_{uv}^{comb};
- 4 source–target distances: bottleneck and Wasserstein distances for H_0 and H_1.

The resulting vector summarizes three signals: the internal topology of the source column, the internal topology of the target column, and the topology of their joint representation. The third signal is the main topological contribution of MetaMatch. It allows the model to represent the geometry of the candidate pair itself, instead of relying only on pooled embedding distances or on distances between two separately built diagrams.

The match/non-match comparison in Figure 2 gives a concrete example of this signal: the non-matching candidate has larger source–target diagram distances and a different combined persistence profile. The goal is not to use topology as a standalone matcher, but to add geometric evidence that complements syntactic and distance-based meta-features.

3.3 Meta-Learner Training

Once the meta-space is built, each candidate pair is represented by a meta-feature vector $\mathbf{m}_{uv} \in \mathbb{R}^{60}$. Let

$$M_{\text{Train}} = \{(\mathbf{m}_{uv}, y_{uv})\}$$

be the training meta-space, where $y_{uv} \in \{0, 1\}$ indicates whether the two columns match.

Before training, the meta-features are normalized so that descriptors with different scales can be used together. The meta-learner is then trained on the syntactic, distance-based, and topological meta-features to produce a scoring model

$$\mathcal{F}(\mathbf{m}_{uv}) = \mathbb{P}(y_{uv} = 1 \mid \mathbf{m}_{uv}).$$

The trained model $\mathcal{F}$ is passed to the inference stage.

3.4 Alignment Inference

For a new table pair, MetaMatch performs **many-to-many alignment**: several source attributes may match several target attributes. It therefore scores all source–target candidate pairs. For each candidate pair (u, v), the trained model $\mathcal{F}$ outputs a match score in $[0, 1]$. A decision threshold τ converts this score into a binary decision:

$$\hat{y}_{uv} = \mathbb{1}\{\mathcal{F}(\mathbf{m}_{uv}) > \tau\}.$$

The final predicted alignment is

$$\widehat{\mathcal{A}} = \{(u, v) \mid \hat{y}_{uv} = 1\}.$$

This stage corresponds to the alignment inference block of Figure 1. The experimental section then specifies the data split, normalization, model choice, and threshold selection.

Table 1: Dataset overview from the Valentine benchmark.

Dataset	# table pairs	# rows	# attributes
TPC-DI	180	7.5k–15k	11–22
ChEMBL	180	7.5k–15k	12–23
OpenData	180	11.6k–23.2k	26–51
WikiData	4	1.8k–5.4k	13–20
Magellan	7	331–66.8k	4–9

4 Experiments

This section evaluates the proposed topology-aware meta-space for schema matching. We study effectiveness against baselines, reduced meta-feature selection, runtime, and meta-feature family ablations.

4.1 Research Questions

Our experimental analysis is organized around the following research questions.

- **RQ1 (Effectiveness and baselines).** How effective is MetaMatch for many-to-many schema matching, which classifier should be used, and how does it compare with established baselines?
- **RQ2 (Meta-feature selection).** Can a two-level selection strategy remove redundant meta-features and retain a reduced, effective meta-space?
- **RQ3 (Efficiency).** How much runtime is saved by the reduced meta-feature set compared with the full meta-space with 60 meta-features and with baselines?
- **RQ4 (Family ablation).** What is the contribution of the syntactic, distance-based, and topological meta-feature families?

4.2 Datasets

We evaluate our framework on the Valentine benchmark [17]. The benchmark contains 551 table pairs from five dataset sources: *TPC-DI*, *ChEMBL*, *WikiData*, *OpenData*, and *Magellan*. These sources cover different schema matching regimes. *TPC-DI* contains moderately structured transactional data; *ChEMBL* contains biomedical attributes and domain-specific vocabulary; *WikiData* contains music-related entities with structural heterogeneity; *OpenData* contains government tables from Canada, the United States, and the United Kingdom; and *Magellan* contains curated entity-matching datasets from domains such as e-commerce and bibliographic data.

4.3 Experimental Setup

For each table pair, we evaluate the complete candidate space: every source attribute is paired with every target attribute. Candidate pairs are represented with the MetaMatch meta-features defined in Section 3.

The evaluation uses six folds built at the table-pair level. All candidates from the same table pair are kept in the same fold. This avoids leakage: the model never sees, during training, another candidate row or reference correspondence from a table pair used for testing. The folds mix the five Valentine sources, so the evaluation tests generalization across domains rather than memorization of one source family.

For MetaMatch, the decision threshold is learned only from training data. In each fold, we choose the threshold that maximizes the mean F1 over the training table pairs, and then apply it unchanged to the test table pairs. Test labels are never used to choose the threshold.

All classifier comparisons and meta-feature selection experiments use the same candidate space and the same six folds. We also repeat the MetaMatch train/test process with ten random seeds over the six folds to check that the results are not driven by one initialization; these seed-level results are reported in the companion artifact.

All experiments are run on a MacBook Pro with a 2.6 GHz 6-core Intel Core i7 processor, 32 GB RAM, and macOS 14.5. Runtime results are therefore reported for this local machine.

We compare MetaMatch with the following baselines on the same table pairs and, when fold-based evaluation is required, on the same six folds: LLMATCH, SMUTF, MagnetoFTGPT (Magneto with fine-tuning and GPT-based augmentation), MagnetoGPT (Magneto with GPT-based augmentation), MagnetoFT (fine-tuned Magneto), Magneto, COMA++, COMA, Similarity Flooding, ISResMat, Cupid, and Distribution Based. We keep the official method names and evaluate all methods as predicted alignments over the candidate space of each table pair.

4.4 Evaluation Metrics

The main effectiveness metric is *F1 all-to-all*. For a table pair p, let G_p be the set of ground-truth correspondences and let A_p be the set of predicted correspondences. For MetaMatch, if s_{uv} is the score assigned to candidate pair (u, v), then the predicted set is

$$A_p(\tau) = \{(u, v) \mid s_{uv} > \tau\},$$

where τ is selected on the training fold only and then applied to the test fold.

For each table pair, we compute

$$\mathrm{TP}_p = |A_p \cap G_p|, \qquad \mathrm{FP}_p = |A_p \setminus G_p|, \qquad \mathrm{FN}_p = |G_p \setminus A_p|.$$

Here, TP means true positives, FP means false positives, and FN means false negatives. Precision, recall, and F1 for table pair p are then

$$\mathrm{Prec}_p = \frac{\mathrm{TP}_p}{\mathrm{TP}_p + \mathrm{FP}_p}, \qquad \mathrm{Rec}_p = \frac{\mathrm{TP}_p}{\mathrm{TP}_p + \mathrm{FN}_p},$$

$$\mathrm{F1}_p = \frac{2\mathrm{TP}_p}{2\mathrm{TP}_p + \mathrm{FP}_p + \mathrm{FN}_p}.$$

True negatives are not used because the candidate space is highly imbalanced and dominated by non-matching pairs. Final reported values are the mean and standard deviation over the 551 table pairs, after computing the metric separately for each table pair so that large candidate spaces do not dominate the evaluation.

4.5 RQ1: Effectiveness and Baselines

Table 2 reports the classifier comparison for MetaMatch. All models are trained on the same meta-space and evaluated on the same folds. Random Forest obtains the best mean F1, precision, and recall. XGBoost and CatBoost are the closest alternatives, whereas Logistic Regression and SVM remain clearly lower. This indicates that the decision boundary in the meta-space is non-linear and that tree-based models are better suited to combine syntactic, distance-based, and topological signals. We therefore use Random Forest for the comparison with baselines.

Table 2: MetaMatch classifier comparison. Values are mean ± standard deviation over table pairs using F1, precision, and recall.

Classifier	F1	Precision	Recall
Random Forest	**0.81** ± 0.24	**0.80** ± 0.26	**0.84** ± 0.24
XGBoost	0.65 ± 0.26	0.73 ± 0.28	0.64 ± 0.30
CatBoost	0.61 ± 0.26	0.65 ± 0.28	0.65 ± 0.31
SVM	0.50 ± 0.25	0.56 ± 0.28	0.55 ± 0.31
Logistic Regression	0.50 ± 0.25	0.56 ± 0.28	0.54 ± 0.31

Table 3: Effectiveness against baselines. Values are mean ± standard deviation over table pairs using F1, precision, and recall.

Method	F1	Precision	Recall
LLMATCH	**0.85** ± 0.25	**0.84** ± 0.28	**0.93** ± 0.18
MetaMatch	0.81 ± 0.24	0.80 ± 0.26	0.84 ± 0.24
MagnetoFTGPT	0.74 ± 0.25	0.76 ± 0.30	0.85 ± 0.19
MagnetoGPT	0.72 ± 0.29	0.70 ± 0.33	0.90 ± 0.15
SMUTF	0.70 ± 0.28	0.72 ± 0.32	0.75 ± 0.28
MagnetoFT	0.61 ± 0.27	0.56 ± 0.29	0.80 ± 0.23
Magneto	0.58 ± 0.29	0.51 ± 0.30	0.84 ± 0.20
COMA++	0.54 ± 0.28	0.73 ± 0.37	0.52 ± 0.30
COMA	0.51 ± 0.27	0.72 ± 0.35	0.48 ± 0.31
Similarity Flooding	0.50 ± 0.35	0.47 ± 0.35	0.68 ± 0.39
ISResMat	0.36 ± 0.29	0.31 ± 0.29	0.80 ± 0.25
Cupid	0.32 ± 0.37	0.43 ± 0.40	0.35 ± 0.42
Distribution Based	0.24 ± 0.25	0.30 ± 0.30	0.24 ± 0.29

Table 3 compares this configuration with the baselines over the same 551 table pairs. MetaMatch reaches a mean F1 of 0.81, with balanced precision and recall. It outperforms symbolic matchers, distance-based baselines, and fine-tuned matching systems, making it the strongest controlled non-LLM configuration in our benchmark.

LLMATCH obtains the highest mean F1. We keep it in the comparison because it is an important recent baseline, but we interpret it separately: it relies on a large external language model whose training data and possible exposure to public benchmark schemas cannot be checked. Apart from this LLM-based method, MetaMatch is the best-performing approach in Table 3. The paired Wilcoxon signed-rank test with Holm correction in Table 4 confirms that the gain over the non-LLM baselines is statistically significant [14, 28].

Figure 3 complements the tables by showing the distribution of F1 across table pairs and the pairwise win rates. This analysis verifies that the result is not driven by a small number of easy pairs. MetaMatch remains competitive across the benchmark and wins against most non-LLM baselines on a large fraction of table pairs.

Figure 4 gives the same analysis by relation type and dataset source. MetaMatch remains strong on Joinable, Semantically-Joinable, Unionable, and View-Unionable pairs, and is especially stable on OpenData, the largest and most heterogeneous source. Magellan and Wikidata contain only 7 and 4 table pairs, respectively, so their dataset-level distributions should be read as qualitative checks. Overall, these results show that MetaMatch is competitive across the benchmark, not only on one subset.

Discussion. Taken together, the classifier comparison, baseline table, distributions, and win-rate matrix show that MetaMatch with Random Forest is the strongest controlled non-LLM configuration in this benchmark. LLMATCH remains higher on average, but we interpret it separately because it relies on an external large language model. The breakdowns by relation type and dataset source indicate that this behavior is not driven by one subset only; MetaMatch stays competitive across the benchmark.

Table 4: Paired Wilcoxon signed-rank test between MetaMatch and each baseline on F1. ΔF1 is MetaMatch minus baseline mean F1 on the 551 paired table pairs. Holm p is corrected for multiple comparisons.

Baseline	ΔF1	Holm p-value	Significant
LLMATCH	-0.05	1.000	No
MagnetoFTGPT	0.06	0.002	Yes
MagnetoGPT	0.08	0.011	Yes
SMUTF	0.10	< 0.001	Yes
MagnetoFT	0.19	< 0.001	Yes
Magneto	0.23	< 0.001	Yes
COMA++	0.26	< 0.001	Yes
COMA	0.30	< 0.001	Yes
Similarity Flooding	0.31	< 0.001	Yes
ISResMat	0.45	< 0.001	Yes
Cupid	0.49	< 0.001	Yes
Distribution Based	0.56	< 0.001	Yes

4.6 RQ2: Meta-feature Selection

We study whether the meta-space can be reduced without losing the signal carried by the topological descriptors. Feature selection is commonly organized into filter, wrapper, embedded, and hybrid families [15]. In preliminary experiments, which will be made available in the companion site, we evaluated several strategies from these families: Mutual Information filters, minimum-redundancy maximum-relevance (mRMR) variants, forward and backward sequential selection, Random Forest and XGBoost importance rankings, SHapley Additive exPlanations (SHAP)-based rankings inspired by recent explainable artificial intelligence feature-selection work [18], and hybrid combinations over several values of k.

The final strategy has two levels. First, we apply Pearson correlation pruning with threshold 0.85. When two meta-features are strongly correlated, we keep the one with the highest Mutual Information with the matching label. This removes repeated measurements of the same evidence while keeping the most informative signal. Second, we rank the remaining meta-features with Random Forest importance and retain the top k meta-features. The value $k = 10$ was selected from the preliminary grid because it gave the best compromise between F1, stability, runtime, and interpretability.

The selected set contains 10 meta-features: 7 syntactic, 2 distance-based, and 1 topological. The syntactic meta-features cover complementary lexical signals: character n-grams, token Jaccard, Jaro–Winkler, LCS ratio, target-name length, and common-suffix ratio. This confirms that names and representative values remain strong evidence in Valentine. The two retained distance-based meta-features are Euclidean and Chebyshev distances between pooled embeddings. They capture two different forms of semantic separation: global displacement in the embedding space and the largest coordinate-level discrepancy. The selected topological descriptor is `tda_h0_entropy_combined`, which summarizes

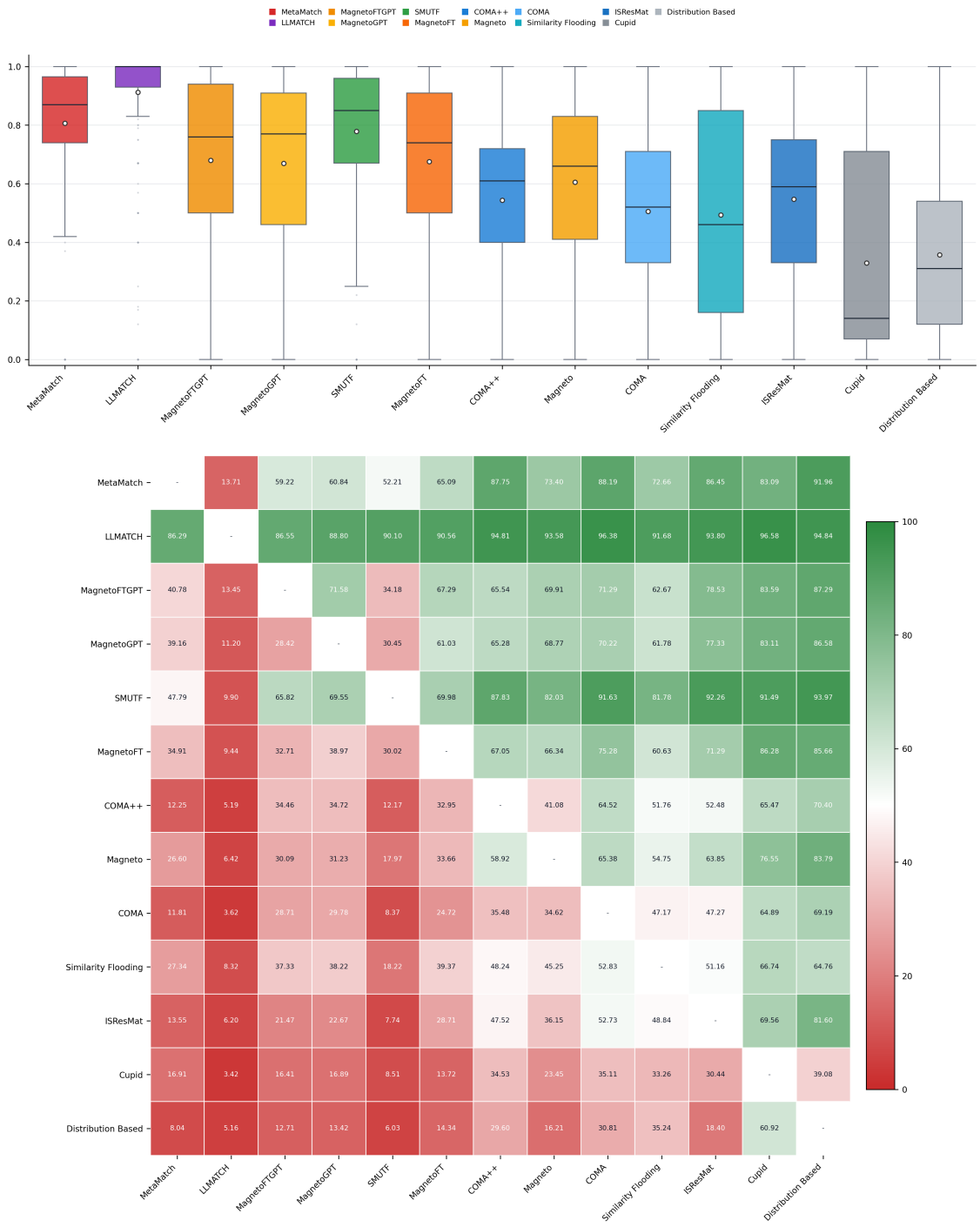

Figure 3: Effectiveness visualization for F1. Top: distribution over table pairs, with the white marker indicating the mean. Bottom: pairwise win-rate matrix; each cell reports how often the row method outperforms the column method, excluding ties.

connected-component persistence entropy on the joint source–target cloud.

Table 5 compares this selected set with the full meta-space. The reduced MetaMatch remains close to the full model, with F1 decreasing only from 0.81 to 0.79, while runtime is greatly reduced. The behavior of the model changes: precision increases from 0.80 to 0.95, while recall decreases from 0.84 to 0.71. The reduced representation is therefore more selective: it outputs fewer uncertain correspondences, but the correspondences it keeps are more reliable.

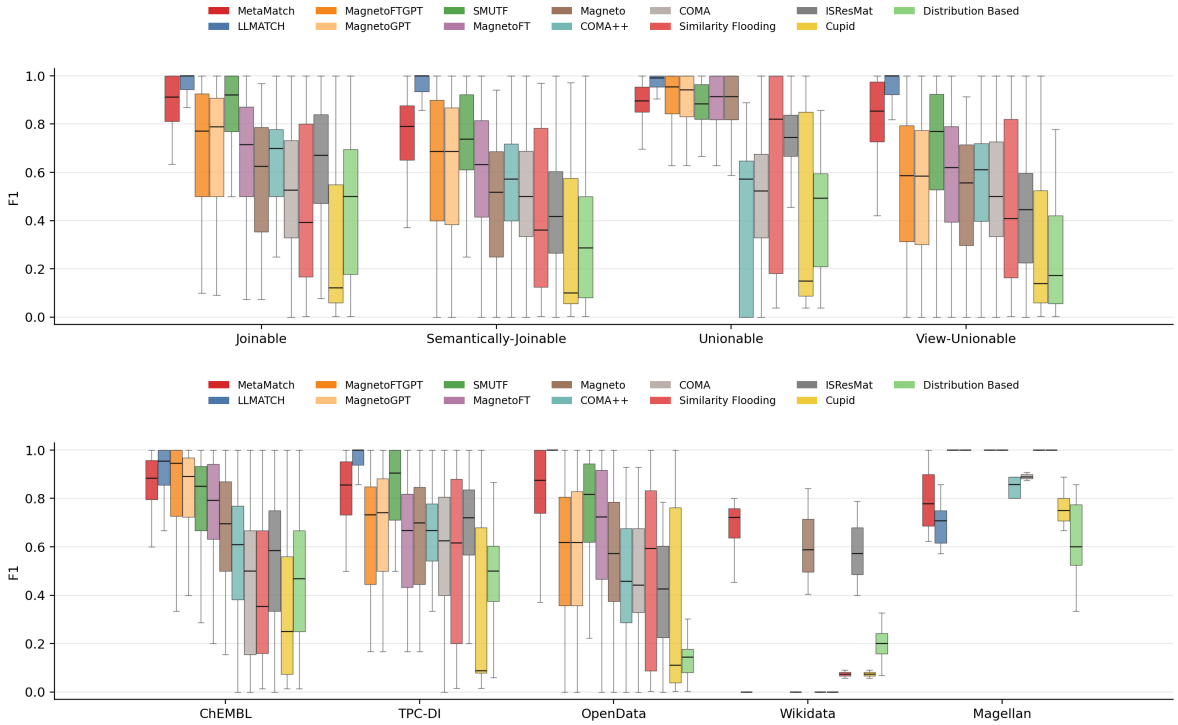

Figure 4: F1 distribution by relation type and dataset source. Each panel compares MetaMatch and baselines on the same pair-level metric used in RQ1.

Discussion. The main observation is that the selected set is not made of one family only. It keeps lexical evidence, two distance-based descriptors, and one topological descriptor. This supports the idea that the three families are not interchangeable. Syntactic meta-features remain essential, distance-based meta-features provide semantic separation, and topology adds a different geometric signal.

The topological result is particularly important. The only retained topological descriptor is not a distance between two separately computed diagrams. It is a combined-cloud descriptor, computed after projecting the two column representations into the same space and building one joint diagram. Thus, topology does not replace classical evidence. It shows that topology complements it with a non-redundant signal, and that this signal comes from the joint source–target construction. The runtime impact of this reduced representation is reported in RQ3.

Table 5: Full MetaMatch versus reduced selected meta-feature set. Meta-feature generation time is estimated from representative table pairs and scaled to all 551 pairs; Random Forest train and test time is reported separately.

Set	# meta-features	F1	Precision	Recall	Generation time (s)	Random Forest train+test (s)	Total time (s)
Full	60	**0.81** ± 0.24	0.80 ± 0.26	**0.84** ± 0.24	66251.24	249.47	66500.71
Reduced	10	0.79 ± 0.18	**0.95** ± 0.16	0.71 ± 0.23	5424.98	195.28	5620.26

4.7 RQ3: Efficiency

For efficiency, we separate preliminary computation from application time. Preliminary computation includes operations performed before final scoring: meta-feature or baseline-feature extraction, training, or fine-tuning when applicable. Application time is the final prediction or scoring step. We report both the full MetaMatch and the reduced MetaMatch selected in RQ2.

This separation is important for a fair reading of the results. For MetaMatch, preliminary time includes meta-feature generation and Random Forest training over the six folds. For SMUTF, it includes its own feature extraction and model training. For MagnetoFT and MagnetoFTGPT, it includes dataset-category fine-tuning before application.

Table 6: Efficiency summary. Preliminary time includes meta-feature or baseline-feature extraction over the 551 table pairs and training or fine-tuning over the six folds when applicable. Application time reports only the prediction/scoring step over the six folds.

Method	Preliminary time (s)	Application time (s)	Total time (s)
Full MetaMatch	66565.13	3.76	66568.89
Reduced MetaMatch	5619.63	0.63	5620.26
SMUTF	59214.97	15.60	59230.57
LLMATCH	0.00	343.76	343.76
MagnetoFTGPT	306456.07	45807.13	352263.20
MagnetoFT	306456.07	9413.56	315869.63
MagnetoGPT	0.00	139243.13	139243.13
Magneto	0.00	10488.01	10488.01
ISResMat	0.00	347.45	347.45
COMA++	0.00	382.48	382.48
COMA	0.00	385.02	385.02
Similarity Flooding	0.00	2875.94	2875.94
Distribution Based	0.00	36318.01	36318.01
Cupid	0.00	128.55	128.55

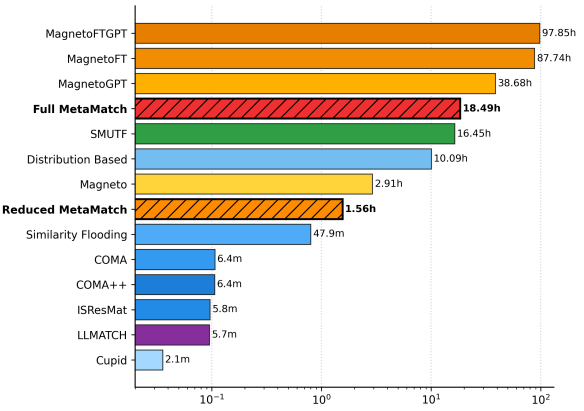

Figure 5: Total runtime over the 551 table pairs, including full and reduced MetaMatch.

Discussion. The main cost of full MetaMatch is not the Random Forest application step, which takes only 3.76 seconds over the six test folds, but the construction of the meta-space. This is expected: topological descriptors require building geometric representations of column-value embeddings and computing persistent-homology summaries. The reduced version changes this trade-off substantially: total runtime decreases from 66568.89 seconds to 5620.26 seconds, an approximately 11.8× speedup, while keeping F1 close to the full model.

Topology-aware evidence is useful, but it must be selected carefully. Lightweight symbolic systems such as Cupid, COMA, and LLMATCH have low measured application time in our setup, while SMUTF and MagnetoFT include substantial preliminary computation because they perform feature extraction, fine-tuning, or dataset-category-specific model preparation. SMUTF also has a longer application step than a single classifier because its prediction stage applies an ensemble of several XGBoost models. Reduced MetaMatch gives the best efficiency/effectiveness trade-off among the MetaMatch variants because it keeps the topological signal selected in RQ2 while avoiding most of the full construction cost.

4.8 RQ4: Family Ablation

Table 7 reports the contribution of the three meta-feature families for both full and reduced MetaMatch. The reduced version keeps the selected meta-features from RQ2 and applies the same family ablation logic inside this reduced representation.

Table 7: Meta-feature family ablation for complete and reduced MetaMatch. Values are mean ± standard deviation over table pairs.

Meta-feature set	Full			Reduced		
	F1	Precision	Recall	F1	Precision	Recall
All meta-features	**.81** ± .24	.80 ± .26	**.84** ± .24	**.79** ± .18	**.95** ± .16	**.71** ± .23
Syntactic	.76 ± .22	.93 ± .20	.68 ± .26	.77 ± .21	.94 ± .19	.68 ± .25
Distance-based	.54 ± .29	.77 ± .33	.49 ± .31	.50 ± .30	.73 ± .34	.44 ± .33
Topological	.55 ± .24	.72 ± .31	.52 ± .27	.05 ± .10	.21 ± .36	.03 ± .09
Syntactic + Distance-based	.78 ± .19	**.95** ± .16	.69 ± .24	.77 ± .19	**.95** ± .17	.69 ± .24
Syntactic + Topological	.79 ± .18	.94 ± .16	.71 ± .23	.78 ± .20	**.95** ± .17	.69 ± .24
Distance-based + Topological	.66 ± .23	.87 ± .24	.59 ± .26	.56 ± .27	.79 ± .31	.50 ± .30

Discussion. The ablation results first show that syntactic meta-features are the strongest individual family, with F1 0.76 in the full representation and 0.77 in the reduced one. This is expected because many benchmark correspondences contain lexical patterns in attribute names and values. However, syntactic evidence alone loses recall compared with the full model, which means that some correct correspondences are missed when the representation only sees lexical similarity.

Distance-based meta-features alone are weaker, with F1 0.54 in the full representation. This supports the motivation of the paper: reducing a column to pooled embedding distances is too coarse for schema matching. Two columns may have similar average embeddings while their value distributions are organized differently. Topological meta-features address this missing information by describing the structure of the value cloud rather than only its average position.

Topology should not be used alone. In the full representation, topology alone reaches F1 0.55. The important result is the gain obtained when topology is combined with other evidence. Syntactic plus topological meta-features reaches F1 0.79, close to the full 0.81 and above syntactic alone. Distance-based plus topological meta-features also improves over distance-based alone, from 0.54 to 0.66.

For reduced MetaMatch, the same pattern remains visible. The reduced model keeps only one topological meta-feature, the combined H0 entropy descriptor, yet syntactic plus topological descriptors reach F1 0.78, close to the full reduced score of 0.79. This confirms the central claim of MetaMatch: topology is not the only source of evidence, but the joint source–target topological representation provides an extra signal that strengthens classical and embedding-based meta-features.

5 Conclusion

We presented MetaMatch, a topology-aware meta-space for schema matching over complete tabular data. The framework combines syntactic, distance-based, and topological meta-features, then learns correspondences with a supervised classifier. Its main method contribution is the introduction of a joint topological representation for candidate column pairs. Beyond computing separate persistence diagrams and diagram distances, MetaMatch projects the two token-level column representations into the same embedding space and builds a single combined diagram that describes their joint shape.

The experiments support this design. Pooled embedding distances alone remain clearly below the full representation, showing that a small set of vector distances is not sufficient to capture the full matching signal. At the same time, topology alone is not intended to replace lexical or semantic evidence. Its value appears when it is combined with them. This is confirmed by the reduced 10-meta-feature model: after correlation pruning and Random Forest importance selection, the only retained topological descriptor is the combined H0 entropy meta-feature. This result gives experimental support to the idea that the way two token clouds merge in a shared space is informative for schema matching.

Overall, MetaMatch shows that topological meta-features can enrich standard schema matching evidence with geometric information that is lost when columns are reduced to single pooled vectors. The reduced version preserves most of the effectiveness of the full representation while greatly lowering the construction cost. Future work will study richer summaries of persistence diagrams, such as lifetime quantiles, persistence images, persistence landscapes, and Betti curves, while controlling their computational

cost. We also plan to extend MetaMatch to ontology matching, where labels, descriptions, graph structure, and logical relations may provide additional signals for the topological meta-space, and to study how the joint topological signal can complement newer language-model-based matchers.

References

[1] David Aumueller, Hong-Hai Do, Sabine Massmann, and Erhard Rahm. 2005. Schema and Ontology Matching with COMA++. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 906–908.

[2] Alexander Bilke and Felix Naumann. 2005. MARLIN: A Machine Learning Approach to Multiple Record Linkage. In *Proceedings of the IEEE International Conference on Data Engineering*. 39–50.

[3] Frédéric Chazal and Bertrand Michel. 2021. An Introduction to Topological Data Analysis: Fundamental and Practical Aspects for Data Scientists. *Frontiers in Artificial Intelligence* 4 (2021), 667963.

[4] Harish Chintakunta, Theofilos Gentimis, Rocio Gonzalez-Diaz, Maria-Jose Jimenez, and Hamid Krim. 2015. An Entropy-Based Persistence Barcode. *Pattern Recognition* 48, 2 (2015), 391–401.

[5] David Cohen-Steiner, Herbert Edelsbrunner, and John Harer. 2007. Stability of Persistence Diagrams. *Discrete & Computational Geometry* 37, 1 (2007), 103–120.

[6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT*. 4171–4186.

[7] Hong-Hai Do and Erhard Rahm. 2002. COMA: A System for Flexible Combination of Schema Matching Approaches. In *Proceedings of the International Conference on Very Large Data Bases*. 610–621.

[8] AnHai Doan, Pedro Domingos, and Alon Halevy. 2003. Learning to Match the Schemas of Data Sources: A Multistrategy Approach. *Machine Learning* 50, 3 (2003), 279–301.

[9] Xin Luna Dong and Divesh Srivastava. 2015. Big Data Integration. *Synthesis Lectures on Data Management* 7, 1 (2015), 1–198.

[10] Xingyu Du, Gongsheng Yuan, Sai Wu, Gang Chen, and Peng Lu. 2024. In Situ Neural Relational Schema Matcher. In *Proceedings of the IEEE International Conference on Data Engineering*. 138–150. doi:10.1109/ICDE60146.2024.00018

[11] Herbert Edelsbrunner, David Letscher, and Afra Zomorodian. 2002. Topological Persistence and Simplification. *Discrete & Computational Geometry* 28, 4 (2002), 511–533.

[12] Grace Fan, Jin Wang, Yuliang Li, Dan Zhang, and Renée J. Miller. 2022. Semantics-aware Dataset Discovery from Data Lakes with Contextualized Column-based Representation Learning. *ArXiv* abs/2210.01922 (2022). doi:10.14778/3587136.3587146

[13] Yuan He, Jiaoyan Chen, Denvar Antonyrajah, and Ian Horrocks. 2022. BERTMap: A BERT-Based Ontology Alignment System. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 5684–5691.

[14] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.

[15] Md Rashedul Islam, Aklima Akter Lima, Sujoy Chandra Das, M. F. Mridha, Akibur Rahman Prodeep, and Yutaka Watanobe. 2022. A Comprehensive Survey on the Process, Methods, Evaluation, and Challenges of Feature Selection. *IEEE Access* 10 (2022), 99595–99632. doi:10.1109/ACCESS.2022.3205618

[16] Jaewoo Kang and Jeffrey F. Naughton. 2003. On Schema Matching with Opaque Column Names and Data Values. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 205–216. doi:10.1145/872757.872783

[17] Christos Koutras, Vasilis Efthymiou, Dimitrios Skoutas, Kostas Stefanidis, and Georgia Koutrika. 2021. Valentine: Evaluating schema matching and entity resolution in dataset discovery. In *Proceedings of the 2021 ACM SIGMOD International Conference on Management of Data*. 2074–2086. doi:10.1145/3448016.3457240

[18] Egor Kraev, Baran Koseoglu, Luca Traverso, and Mohammed Topiwalla. 2024. Shap-Select: Lightweight Feature Selection Using SHAP Values and Regression. *arXiv preprint arXiv:2410.06815* (2024). arXiv:2410.06815

[19] Yurong Liu, Eduardo Pena, Aécio Santos, Eden Wu, and Juliana Freire. 2025. Magneto: Combining Small and Large Language Models for Schema Matching. *Proceedings of the VLDB Endowment* 18, 8 (2025), 2681–2694. doi:10.14778/3742728.3742757

[20] Jayant Madhavan, Philip A. Bernstein, and Erhard Rahm. 2001. Generic Schema Matching with Cupid. In *Proceedings of the International Conference on Very Large Data Bases*. 49–58. https://www.vldb.org/conf/2001/P049.pdf

[21] Emanuela Merelli, Matteo Rucco, Peter M. A. Sloot, and Luca Tesei. 2015. Topological Characterization of Complex Systems: Using Persistent Entropy. *Entropy* 17, 10 (2015), 6872–6892.

[22] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. 2018. Deep Learning for Entity Matching: A Design Space Exploration. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 19–34.

[23] Marcel Parciak, Brecht Vandevoort, Frank Neven, Liesbet M. Peeters, and Stijn Vansummeren. 2024. Schema Matching with Large Language Models: an Experimental Study. *arXiv preprint arXiv:2407.11852* (2024). https://arxiv.org/abs/2407.11852

[24] Erhard Rahm and Philip A. Bernstein. 2001. A Survey of Approaches to Automatic Schema Matching. *The VLDB Journal* 10, 4 (2001), 334–350.

[25] Fahad Ahmed Satti, Musarrat Hussain, Sungyoung Lee, and TaeChoong Chung. 2022. Significance of Syntactic Type Identification in Embedding Vector based Schema Matching. In *Proceedings of the 16th International Conference on Ubiquitous Information Management and Communication (IMCOM)*. 1–6. doi:10.1109/imcom53663.2022.9721780

[26] Eitam Sheetrit, Menachem Brief, Moshik Mishaeli, and Oren Elisha. 2024. ReMatch: Retrieval Enhanced Schema Matching with LLMs. *arXiv preprint arXiv:2403.01567* (2024). https://arxiv.org/abs/2403.01567

[27] Sha Wang, Yuchen Li, Hanhua Xiao, Bing Tian Dai, Roy Ka-Wei Lee, Yanfei Dong, and Lambert Deng. 2025. LLMATCH: A Unified Schema Matching Framework with Large Language Models. *arXiv preprint arXiv:2507.10897* (2025). https://arxiv.org/abs/2507.10897

[28] Frank Wilcoxon. 1945. Individual Comparisons by Ranking Methods. *Biometrics Bulletin* 1, 6 (1945), 80–83.

[29] Alexandros Zeakis, G. Papadakis, Dimitrios Skoutas, and Manolis Koubarakis. 2023. Pre-trained Embeddings for Entity Resolution: An Experimental Analysis. *Proc. VLDB Endow.* 16 (2023), 2225–2238. doi:10.14778/3598581.3598594

[30] Yu Zhang, Mei Di, Haozheng Luo, Chenwei Xu, and Richard Tzong-Han Tsai. 2024. SMUTF: Schema Matching Using Generative Tags and Hybrid Features. *arXiv preprint arXiv:2402.01685* (2024). https://arxiv.org/abs/2402.01685

[31] Afra Zomorodian and Gunnar Carlsson. 2005. Computing Persistent Homology. *Discrete & Computational Geometry* 33, 2 (2005), 249–274.